```
def mkName(name: String): Either[String, Name] =
  if (name == "" || name == null) Left("Name is empty.")
  else Right(new Name(name))

def mkAge(age: Int): Either[String, Age] =
  if (age < 0) Left("Age is out of range.")
  else Right(new Age(age))

def mkPerson(name: String, age: Int): Either[String, Person] =
  mkName(name).map2(mkAge(age))(Person(_, _))
```

EXERCISE 4.8

In this implementation, `map2` is only able to report one error, even if both the name and the age are invalid. What would you need to change in order to report *both* errors? Would you change `map2` or the signature of `mkPerson`? Or could you create a new data type that captures this requirement better than `Either` does, with some additional structure? How would `orElse`, `traverse`, and `sequence` behave differently for that data type?

4.5 Summary

In this chapter, we noted some of the problems with using exceptions and introduced the basic principles of purely functional error handling. Although we focused on the algebraic data types `Option` and `Either`, the bigger idea is that we can represent exceptions as ordinary values and use higher-order functions to encapsulate common patterns of handling and propagating errors. This general idea, of representing effects as values, is something we'll see again and again throughout this book in various guises.

We don't expect you to be fluent with all the higher-order functions we wrote in this chapter, but you should now have enough familiarity to get started writing your own functional code complete with error handling. With these new tools in hand, exceptions should be reserved only for truly unrecoverable conditions.

Lastly, in this chapter we touched briefly on the notion of a *non-strict* function (recall the functions `orElse`, `getOrElse`, and `Try`). In the next chapter, we'll look more closely at why non-strictness is important and how it can buy us greater modularity and efficiency in our functional programs.